DPFNet-HMC: Enhancing Particle Diversity in Differentiable Particle Filters via Amortized Transport and Hamiltonian Rejuvenation

Lixin Tu
School of Engineering
The University of British Columbia
Kelowna, BC, Canada V1V1V7
lixin.tu@ubc.ca

Abstract

Sequential Monte Carlo (SMC), or particle filters (PFs), are a standard tool for nonlinear, non-Gaussian state estimation. However, they suffer from weight collapse and rely on non-differentiable resampling steps, limiting their use in deep learning. We propose DPFNet-HMC, an end-to-end differentiable particle filter (DPF) that combines an amortized neural transport map with Hamiltonian Monte Carlo (HMC) rejuvenation. The transport stage uses a residual network (GradNet) to smoothly move particles toward high-likelihood regions, bypassing computationally expensive optimal transport (OT) solvers. Next, HMC restores particle diversity by using exact likelihood gradients to explore the local geometry of the posterior. After rejuvenation, weights are reset to uniform values to maintain differentiability. Evaluated on a high-dimensional, nonlinear state-space model (SSM) benchmark, DPFNet-HMC significantly improves point estimates while maintaining a stable effective sample size (ESS). A lighter variant, DPF-GradNet, achieves comparable accuracy while running much faster. These results demonstrate that learned transport efficiently replaces iterative OT solvers, and HMC provides a mathematically principled diversity mechanism when particle collapse is severe. Code is available at `https://github.com/Lixin-TU/mlcoe_26/tree/main/Q2_update`.

1 Introduction

SMC methods, or PFs, approximate posterior distributions with weighted samples [1]. In high-dimensional spaces, however, these filters frequently experience weight degeneracy, where nearly all probability mass concentrates on a single particle [2]. Furthermore, the traditional discrete resampling step used to discard low-weight particles breaks the computational graph, preventing gradient flow and end-to-end training.

Two primary approaches attempt to solve this. Stochastic particle flow (SPF) moves particles continuously from a prior to a posterior distribution using either a stochastic differential equation (SDE) or a deterministic flow map [3]. Alternatively, DPFs replace discrete resampling with smooth, differentiable transformations often grounded in OT theory [4]. However, continuous-flow methods can still suffer from mode collapse when particles are aggressively pushed toward a single high-probability region. HMC addresses this specific vulnerability by utilizing gradient-informed dynamics to rigorously explore the posterior space, thereby maintaining diversity [5].

We introduce DPFNet-HMC, a novel filtering architecture synthesizing amortized neural transport and HMC rejuvenation. A GradNet-based residual transport map deterministically shifts particles toward high-likelihood regions. An HMC step then utilizes exact gradients to refresh the particle

Submitted to MLCOE TSRL Internship 2026 (Question 2).

states, preventing collapse. Resetting weights to a uniform distribution yields a fully differentiable pipeline. The rest of this paper is organized as follows: Section 2 reviews the preliminaries on stochastic particle flow, differentiable particle filters, and Hamiltonian Monte Carlo. Section 3 formulates the non-linear, non-Gaussian state-space tracking problem. Section 4 details the proposed DPFNet-HMC methodology. Section 5 presents the experimental setup, comparative analysis, and stability diagnostics. Finally, Section 6 concludes the paper.

2 Preliminaries

2.1 Stochastic Particle Flow

Motivation SPF filters continuously morph a prior distribution into a posterior, bypassing the numerical singularities common in optimal transport. A generalized log-homotopy path is often introduced to reduce the condition number of the Hessian of the posterior density, thereby mitigating the stiffness of the underlying SDEs and reducing state-estimation errors [3].

Mathematical expression To transition from a prior $p_0(x)$ to a posterior $p_1(x)$ given a likelihood $h(x)$, we introduce a continuous parameter $\lambda \in [0, 1]$. The log-homotopy is a time-varying combination of the log-prior and log-posterior:

$$\log p(x, \lambda) = \alpha(\lambda) \log p_0(x) + \beta(\lambda) \log p_1(x) - \log \Gamma(\lambda) \tag{1}$$

where $\alpha(\lambda)$ and $\beta(\lambda)$ are transition functions, and $\Gamma(\lambda)$ normalizes the density. The particles $x(\lambda)$ evolve via the SDE:

$$dx = f(x, \lambda)d\lambda + q(x, \lambda)dw_\lambda \tag{2}$$

where $f(x, \lambda) \in \mathbb{R}^n$ is the drift, $q(x, \lambda)$ is the diffusion matrix, and w_λ is Brownian motion.

Algorithm 1: Stochastic Particle Flow Update

Input: Prior particles $\{x_i(0)\}_{i=1}^N$, Prior p_0, Likelihood h, Diffusion matrix Q, Step size $\Delta\lambda$
Output: Posterior particles $\{x_i(1)\}_{i=1}^N$

Function *SPFUpdate($\{x_i(0)\}$, p_0, h, Q)*:
  Solve 1-D boundary value problem to find optimal transition functions $\alpha^*(\lambda), \beta^*(\lambda)$;
  for $\lambda = 0$ **to** 1 $\Delta\lambda$ **do**
    for $i = 1$ **to** N **do**
      $f_i \leftarrow K_1(\nabla_x \log p_0(x_i)) + K_2(\nabla_x \log h(x_i))$; `// Calculate drift`
      $dw \sim \mathcal{N}(0, \Delta\lambda I)$;
      $x_i(\lambda + \Delta\lambda) \leftarrow x_i(\lambda) + f_i\Delta\lambda + q(x_i, \lambda)dw$; `// Euler-Maruyama step`
    end
  end
  return $\{x_i(1)\}_{i=1}^N$
end

2.2 Differentiable Particle Filter

Motivation Because traditional resampling is non-differentiable, DPFs substitute it with differentiable transport plans, enabling standard backpropagation.

Mathematical expression Resampling is formulated as an OT problem [6]. We find a transport matrix P moving mass from a uniformly weighted distribution α_N to a non-uniformly weighted target β_N. We minimize an entropy-regularized 2-Wasserstein distance for smooth gradients:

$$\mathcal{W} * 2, \epsilon^2 (\alpha_N, \beta_N) = \min * P \in \mathcal{S}(a, b) \sum * i, j = 1^{N_p} p * i, j \left(c_{i,j} + \epsilon \log \frac{p_{i,j}}{a_i b_j} \right) \tag{3}$$

where $c_{i,j} = \left\| X_t^i - X_t^j \right\|^2$ is the cost matrix, and $\epsilon > 0$ is a regularization parameter. The resulting coupling P_ϵ^{OT} is found via Sinkhorn iterations. New particles $\tilde{X}_t^i$ are generated deterministically

using a barycentric projection:

$$\tilde{X} * t^i = N_p \sum * k = 1^{N_p} p^{OT}_{\epsilon,i,k} X^k_t \tag{4}$$

Algorithm 2: Differentiable OT Resampling

Input: Particles X_t, Weights W_t, Regularization ϵ, Iterations K

Output: Resampled equally-weighted particles $\tilde{X}_t$

Function *OTResample(X_t, W_t, ϵ)*:

  Compute pairwise squared Euclidean cost matrix C where $c_{i,j} = \|X^i_t - X^j_t\|^2$;

  Initialize scaling vectors $u = \mathbf{1}/N_p$, $v = \mathbf{1}/N_p$;

  $K_{\text{gibbs}} \leftarrow \exp(-C/\epsilon)$;

  for $k = 1$ **to** K **do**

    $v \leftarrow W_t/(K^T_{\text{gibbs}} u)$; `// Sinkhorn alternating projections`

    $u \leftarrow (\mathbf{1}/N_p)/(K_{\text{gibbs}} v)$;

  end

  $P^{OT}_\epsilon \leftarrow \text{diag}(u) K_{\text{gibbs}} \text{diag}(v)$; `// Optimal Transport Matrix`

  $\tilde{X}_t \leftarrow N_p P^{OT}_\epsilon X_t$; `// Barycentric projection`

  return $\tilde{X}_t$

end

2.3 Hamiltonian Monte Carlo with Invertible Flows

Motivation HMC leverages the gradient of a target log-density for highly efficient parameter space exploration. By combining invertible normalizing flows with differentiable OT resampling, a standard PF can be integrated into an HMC framework to perform Bayesian inference on static model parameters.

Mathematical expression To sample parameters θ from the posterior $p(\theta \mid y_{1:T})$, HMC uses the exact gradient:

$$\nabla_\theta \log p\left(\theta \mid y_{1:T}\right) = \nabla_\theta \log \hat{p}\left(y_{1:T} \mid \theta\right) + \nabla_\theta \log p(\theta) \tag{5}$$

To ensure the log-marginal-likelihood $\log \hat{p}(y_{1:T} \mid \theta)$ is differentiable, particles are proposed using conditional invertible normalizing flows. Because the transformation is invertible and resampling is differentiable via Sinkhorn OT, the particle weights remain smooth functions of θ, preserving the necessary gradient flow for Hamiltonian dynamics.

3 Problem Statement

Consider a general discrete-time, non-linear, and non-Gaussian state-space model (SSM). Let $t \in \mathbb{N}$ denote the discrete time index. The hidden state of the system is denoted by $x_t \in \mathbb{R}^{d_x}$, and the corresponding observation (or measurement) is denoted by $y_t \in \mathbb{R}^{d_y}$. The system dynamics and the observation mechanism are formally defined by the following discrete-time equations:

$$x_t = f\left(x_{t-1}\right) + v_t \tag{6}$$

$$y_t = h\left(x_t\right) + e_t \tag{7}$$

Here, $f(\cdot)$ and $h(\cdot)$ are potentially nonlinear functions that represent the state transition and measurement processes, respectively. The terms v_t and e_t denote process and measurement noise, assumed independent and following non-Gaussian distributions.

Equivalently, in probabilistic terms, the model is fully characterized by an initial state distribution, a transition density, and an observation (emission) density:

- Initial State: $x_0 \sim p\left(x_0\right)$
- Transition Density: $x_t \sim p\left(x_t \mid x_{t-1}\right)$

Algorithm 3: HMC Inference within Differentiable PF

Input: Observations $y_{1:T}$, Initial parameters θ_0, Leapfrog steps L, Step size ϵ, MCMC iterations M

Output: MCMC Chain of parameters $\{\theta_m\}_{m=1}^{M}$

Function *HMCDPFInference*($y_{1:T}$, θ_0)**:**

- $\theta \leftarrow \theta_0$;
- **for** $m = 1$ **to** M **do**
  - `// Forward & Backward Pass via Auto-Differentiation`
  - $\log \hat{p} \leftarrow \text{DifferentiableParticleFilter}(y_{1:T}, \theta, \mathcal{G}_\phi, \text{OTResample})$;
  - $g \leftarrow \nabla_\theta(\log \hat{p}(y_{1:T} \mid \theta) + \log p(\theta))$; `// Exact gradient via backprop`
  - `// Hamiltonian Dynamics`
  - Sample momentum $r \sim \mathcal{N}(0, I)$;
  - $\theta_{prop}, r_{prop} \leftarrow \text{Leapfrog}(\theta, r, g, \epsilon, L)$;
  - `// Metropolis Acceptance`
  - $\alpha \leftarrow \exp(H(\theta, r) - H(\theta_{prop}, r_{prop}))$;
  - **if** $U(0,1) < \min(1, \alpha)$ **then**
    - $\theta \leftarrow \theta_{prop}$;
  - **end**
  - Record $\theta_m \leftarrow \theta$;
- **end**
- **return** $\{\theta_m\}_{m=1}^{M}$

end

- Observation Density: $y_t \sim p\left(y_t \mid x_t\right)$

Given a sequence of observations up to time t, denoted as $y_{1:t} = \{y_1, y_2, \ldots, y_t\}$, the primary objective of Bayesian sequential filtering is to estimate the marginal posterior probability density function of the current hidden state, $p\left(x_t \mid y_{1:t}\right)$.

According to Bayes' theorem, this posterior can be recursively updated in two steps:

Step 1: Prediction

$$p\left(x_t \mid y_{1:t-1}\right) = \int p\left(x_t \mid x_{t-1}\right) p\left(x_{t-1} \mid y_{1:t-1}\right) dx_{t-1} \tag{8}$$

Step 2: Update

$$p\left(x_t \mid y_{1:t}\right) = \frac{p\left(y_t \mid x_t\right) p\left(x_t \mid y_{1:t-1}\right)}{\int p\left(y_t \mid x_t\right) p\left(x_t \mid y_{1:t-1}\right) dx_t} \tag{9}$$

For linear models subject to Gaussian noise, these integrals yield closed-form Gaussian solutions, famously known as the Kalman Filter. However, in the general non-linear, non-Gaussian case presented here, the integrals in the prediction and update steps are analytically intractable. This exact intractability directly motivates the use of approximate inference methods, such as PFs, which represent the posterior distribution as a set of weighted samples to solve non-linear sequential state estimation problems.

4 Proposed method: DPFNet-HMC

Standard Particle Filters often suffer from weight degeneracy, where most particles end up with near-zero weights, and from sample impoverishment caused by traditional discrete resampling, which duplicates high-weight particles. To address these issues, we propose DPFNet-HMC, a fully differentiable pipeline that replaces traditional resampling with an amortized neural transport mechanism followed by HMC transitions. At each time step, particles are first propagated forward using the underlying transition model, and their importance weights are updated based on the observation log-likelihood. Instead of discarding low-weight particles, we utilize a residual transport map to continuously move all particles. This particle flow comprises a deterministic mean-pull toward

the ensemble's weighted mean and a non-linear neural displacement computed by a multi-layer perceptron (GradNet), which takes the concatenated particle states and weights as input and outputs a displacement vector.

While the neural transport efficiently maps particles to high-probability regions, we introduce an HMC rejuvenation step immediately afterward to prevent mode collapse and mathematically guarantee rigorous local exploration. Using automatic differentiation to obtain exact gradients of the observation log-probability, we simulate Hamiltonian dynamics via Leapfrog integration to propose new particle states, which are then accepted or rejected using a Metropolis-Hastings criterion. Because the particles have been explicitly relocated to represent the unweighted posterior distribution, the pipeline concludes by resetting all particle weights to a uniform distribution. This hybrid approach ensures improved particle diversity while bypassing the non-differentiable and lossy nature of discrete resampling, as summarized in Algorithm 4.

Algorithm 4: DPFNet-HMC Update Step

Input: Particles X_{t-1}, log-weights $\log W_{t-1}$, observation y_t
Output: Updated particles X_t, uniform log-weights $\log W_t$

Function *Step*$(X_{t-1}, \log W_{t-1}, y_t)$**:**
  `// 1. Prediction and Update`
  $X_{t|t-1} \sim p(X_t|X_{t-1})$; `// Transition model prior`
  $\log W_{t|t-1} \leftarrow \log W_{t-1} + \log p(y_t|X_{t|t-1})$; `// Observation likelihood`
  $W \leftarrow \text{Normalize}(\exp(\log W_{t|t-1}))$;
  `// 2. Amortized Neural Transport (Particle Flow)`
  $\mu \leftarrow \sum(W \cdot X_{t|t-1})$; `// Weighted mean`
  $D_{pull} \leftarrow \lambda_{pull}(\mu - X_{t|t-1})$; `// Deterministic mean-pull`
  $D_{net} \leftarrow \lambda_{net} \tanh(\text{GradNet}([X_{t|t-1}, W]))$; `// Neural displacement`
  $X_{trans} \leftarrow X_{t|t-1} + D_{pull} + D_{net}$;
  `// 3. HMC Rejuvenation`
  for $s \leftarrow 1$ **to** S **do**
    Sample auxiliary momentum $P \sim \mathcal{N}(0, I)$;
    `// Simulate Hamiltonian dynamics using exact gradients`
    $X_{prop}, P_{prop} \leftarrow \text{Leapfrog}(X_{trans}, P, \nabla_X \log p(y_t|X))$;
    $X_{trans} \leftarrow \text{Metropolis-Hastings-Accept}(X_{prop}, X_{trans})$;
  end
  $X_t \leftarrow X_{trans}$;
  `// 4. Weight Reset`
  $\log W_t \leftarrow -\log N$; `// Reset to uniform distribution`
  return $X_t, \log W_t$
end

5 Experiments

5.1 Experiment Setup

5.1.1 High-Dimensional Non-linear Dataset

To evaluate the performance and scalability of our proposed methods, we extend the classic univariate non-linear State Space Model (SSM) [7] to a high-dimensional setting. Let the state vector be $X_n \in \mathbb{R}^d$ and the observation vector be $Y_n \in \mathbb{R}^d$ (or $m < d$). The dynamics and observations follow an element-wise mapping defined as:

State Transition Model For each dimension $i \in \{1, \ldots, d\}$, the state evolves as:

$$X_{n,i} = \frac{X_{n-1,i}}{2} + 25\frac{X_{n-1,i}}{1 + X_{n-1,i}^2} + 8\cos(1.2n) + V_{n,i} \tag{10}$$

where $V_n \sim \mathcal{N}\left(0, \sigma_V^2 \mathbf{I}_d\right)$ is the i.i.d. Gaussian process noise.

Observation Model The observations are generated via a quadratic mapping that discards the sign information of the states:

$$Y_{n,i} = \frac{X_{n,i}^2}{20} + W_{n,i} \tag{11}$$

where $W_n \sim \mathcal{N}\left(0, \sigma_W^2 \mathbf{I}_d\right)$ is the i.i.d. Gaussian measurement noise.

The state transition contains a rational function and a time-varying cosine modulation. Even with Gaussian noise, the resulting posterior distribution $p\left(x_{1:T} \mid y_{1:T}\right)$ is non-Gaussian. Fig. 1 below shows a visualization of a 10-dimensional system ($d = 10$) over $T = 100$ time steps using the synthesized data.

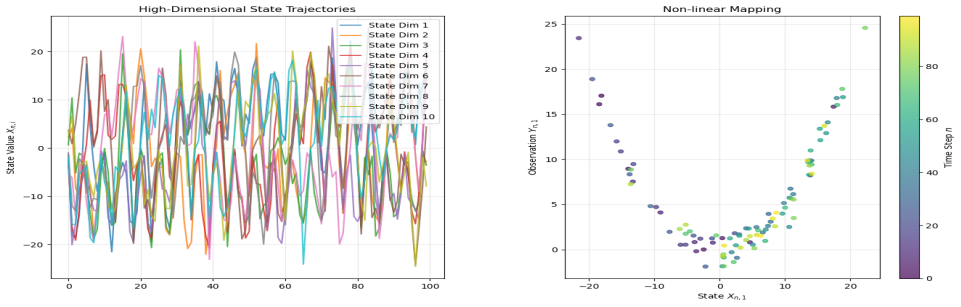

Figure 1: Visualization of the synthesized datasets: high-dimensional (left) and nonlinear (right).

We generated two sets of observations $y_{1:100}$ with $\sigma_V^2 = \sigma_W^2 = 10$, and $\sigma_V^2 = 10$ and $\sigma_W^2 = 1$ for the following experiments.

5.1.2 Baselines and Ablations

To comprehensively evaluate our proposed methods on non-linear, high-dimensional tasks in terms of accuracy, stability, and efficiency, we benchmark against five established filtering algorithms and two of our proposed variants:

SPFSM [3]: A stochastic particle flow filter that mitigates numerical stiffness by explicitly minimizing the condition number of the posterior Hessian matrix.

IPFPF [8]: The Local Exact Daum-Huang (LEDH) variant, representing traditional deterministic invertible particle flows without neural components.

DPFS [9]: A differentiable particle filter utilizing "soft-resampling," which interpolates between importance-weighted and uniform distributions to preserve end-to-end gradient flow.

DPFOT [9]: A state-of-the-art differentiable filter that formulates resampling as an Optimal Transport (OT) problem, solved iteratively via the entropy-regularized Sinkhorn algorithm.

DPFHS [10]: A foundational neural state-space model that learns observation features but retains standard, non-differentiable hard systematic resampling.

DPFOT-HMC (Proposed Ablation): An extension of DPFOT that appends Hamiltonian Monte Carlo (HMC) transitions after OT resampling, utilizing gradient-guided exploration to prevent particle depletion.

DPF-GradNet (Proposed Ablation): An amortized OT approach [11] that replaces the computationally expensive Sinkhorn algorithm with a Conditional GradNet. It directly predicts the optimal transport map from the particle context, significantly reducing computational overhead.

5.1.3 Evaluation Metrics

To comprehensively assess the proposed filtering algorithms, we evaluate their performance against the baselines across three dimensions: accuracy, stability, and computational efficiency.

Accuracy metrics quantify the precision of the state estimation.

- Root Mean Square Error (RMSE): Measures the average point-estimation error across the trajectory, defined as:

$$RMSE = \sqrt{\frac{1}{T}\sum_{t=1}^{T} \|\hat{x}_t - x_t\|_2^2} \tag{12}$$

  where $\hat{x}_t$ is the posterior mean estimate (the weighted sum of particles) and x_t is the true hidden state at time t. A lower RMSE indicates that the tracking algorithm faithfully follows the true system dynamics.

- Empirical Coverage: evaluates the statistical validity of the posterior distribution, defined as:

$$\text{Coverage} = \frac{1}{T}\sum_{t=1}^{T} \mathbb{I}\left(x_t \in \mathcal{C}_t^{1-\alpha}\right) \tag{13}$$

  where $\mathcal{C}_t^{1-\alpha}$ is the estimated $(1-\alpha)$ credible interval (typically 95%) and $\mathbb{I}(\cdot)$ is the indicator function. Coverage significantly below the target interval indicates overconfidence (underestimating uncertainty), while coverage near 100% suggests the filter is excessively conservative.

Stability metrics diagnose the internal health of the particle ensemble and its robustness against weight degeneracy.

- Effective Sample Size (ESS): Quantifies the diversity of the particle hypotheses, defined as:

$$ESS_t = \left(\sum_{i=1}^{N_p} \left(W_t^i\right)^2\right)^{-1} \tag{14}$$

  where W_t^i represents the normalized importance weight of the i-th particle. A high and stable ESS indicates a healthy ensemble, whereas a drop toward 1 signifies severe weight degeneracy (a single particle dominating), which typically triggers resampling to prevent filter collapse.

Efficiency metrics quantify the physical computational resources during inference.

- Runtime (t_{total}): The total wall-clock time required to complete an inference.

5.1.4 Hardware and Software Configuration

All numerical experiments and performance benchmarks were conducted on a local workstation equipped with an Intel Core i9-14900KF processor (3.20 GHz) and 32.0 GB of RAM, running a 64-bit operating system. To accelerate highly parallelizable matrix operations, an NVIDIA GeForce RTX 4080 SUPER GPU was utilized.

The filtering algorithms and neural network components were implemented using the TensorFlow framework. To balance computational efficiency with the numerical precision required for unnormalized particle weights and log-likelihood accumulations, all tensor operations were executed using the standard single-precision floating-point format (float32).

5.2 Comparative Analysis

Table 1 presents the quantitative comparison across the two noise scenarios, revealing a clear trade-off between point-estimation accuracy and uncertainty calibration. The proposed DPF-GradNet and DPFNet-HMC consistently achieve the lowest RMSE (approximately 25) in both scenarios, substantially outperforming all baseline models. This confirms that learning an amortized transport map, or augmenting OT resampling with HMC transitions, significantly improves state-estimation accuracy. However, classical particle flow methods, namely SPFSM and IPFPF, attain the highest empirical coverage (around 93%), indicating well-calibrated uncertainty estimates despite their

higher RMSE. In contrast, pure OT-based resampling (DPFOT) tends to over-concentrate particles and underestimate posterior uncertainty, with coverage dropping to 26.1% under the high-noise scenario ($\sigma_W^2 = 10$). While integrating HMC (DPFOT-HMC) partially alleviates particle collapse by improving coverage to 32.9%, the hard systematic resampling baseline (DFPHS) still suffers a complete collapse in the low-noise regime ($\sigma_W^2 = 1$), dropping to 6.8% coverage.

Beyond estimation accuracy, the proposed methods demonstrate notable advantages in computational efficiency. Validating the fundamental benefit of amortized inference, DPF-GradNet achieves top-tier accuracy while running in under $0.4\,\mathrm{s}$. This approach is nearly an order of magnitude faster than DPFNet-HMC, which requires 2.8 to $3.0\,\mathrm{s}$, and it completely bypasses the computational bottleneck of the iterative Sinkhorn algorithm utilized in DPFOT, which takes approximately $6.3\,\mathrm{s}$ per inference.

Table 1: Comparative analysis across two scenarios.
(Results are reported as mean $\pm$ std over 5 random seeds. **Bold** indicates the best result and underline indicates the second best. † denotes the proposed method.)

Scenario	Method	RMSE ↓	Coverage (%) ↑	Runtime (s) ↓
$\sigma_W^2 = 1$	SPFSM [3]	29.42 ± 0.07	**93.42 ± 0.54**	1.53 ± 0.03
	IPFPF [8]	29.46 ± 0.05	93.14 ± 0.15	0.84 ± 0.07
	DPFS [9]	32.43 ± 1.06	58.62 ± 0.77	**0.29 ± 0.04**
	DPFOT [9]	31.46 ± 0.31	52.16 ± 0.79	6.28 ± 0.18
	DFPHS [10]	38.04 ± 1.39	6.76 ± 1.75	0.31 ± 0.06
	DPFOT-HMC	31.64 ± 1.25	50.34 ± 0.96	7.54 ± 0.19
	DPF-GradNet	**24.99 ± 0.82**	72.18 ± 1.33	0.38 ± 0.02
	DPFNet-HMC†	25.42 ± 0.37	71.84 ± 1.03	2.82 ± 0.08
$\sigma_W^2 = 10$	SPFSM [3]	29.42 ± 0.07	**93.42 ± 0.54**	1.55 ± 0.04
	IPFPF [8]	29.46 ± 0.05	93.14 ± 0.15	0.79 ± 0.05
	DPFS [9]	31.93 ± 1.59	65.82 ± 3.40	**0.26 ± 0.01**
	DPFOT [9]	29.84 ± 0.35	26.14 ± 1.57	6.35 ± 0.11
	DFPHS [10]	32.56 ± 1.49	41.46 ± 3.40	0.32 ± 0.06
	DPFOT-HMC	29.44 ± 0.19	32.88 ± 2.29	7.81 ± 0.37
	DPF-GradNet	24.58 ± 0.78	72.26 ± 1.17	0.39 ± 0.02
	DPFNet-HMC†	**24.55 ± 0.51**	72.46 ± 1.51	3.03 ± 0.14

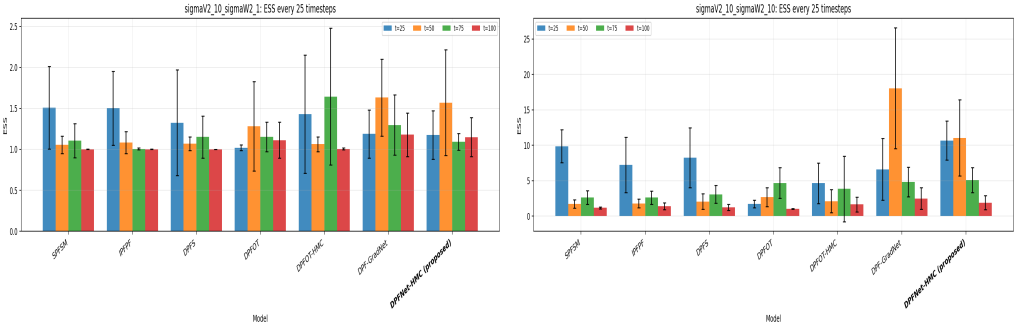

Figure 2: ESS across time steps for two scenarios for each model (Left:$\sigma_W^2 = 1$; Right: $\sigma_W^2 = 10$).

Additionally, Fig. 2 illustrates the dynamics of the Effective Sample Size (ESS), providing deeper insights into particle stability. In the low-noise setting ($\sigma_W^2 = 1$), all evaluated models maintain a relatively stable ESS between 1.0 and 1.6, suggesting adequately preserved particle diversity. However, the high-noise setting ($\sigma_W^2 = 10$) reveals severe vulnerabilities in the baselines: traditional flow methods degrade sharply over time, whereas DPFOT maintains a consistently low ESS throughout the sequence. Although DPF-GradNet occasionally yields high ESS, it suffers from substantial variance. In contrast, our primary model, DPFNet-HMC, demonstrates highly competitive and stable

ESS across all timesteps and noise conditions. This indicates that the hybrid architecture successfully achieves a robust balance between preserving particle diversity and maintaining computational tractability. Further in-depth analysis of stability diagnostics is provided in the following subsection.

5.3 Stability diagnostics

Fig. 3 presents stability diagnostics across 100 iterations for two noise scenarios, reporting mean flow magnitude (L2 displacement) and Jacobian condition number for flow-based models.

Flow Magnitude DPFOT and DPFOT-HMC exhibit consistently large and highly volatile flow magnitudes in both scenarios, with L2 displacements oscillating between roughly 2555 $\left(\sigma_W^2 = 1\right)$ and $30 - 85$ $\left(\sigma_W^2 = 10\right)$, indicating that their particle transport steps involve aggressive, potentially destabilizing displacements. Notably, the two curves track closely, suggesting that adding HMC does not substantially reduce the magnitude of the underlying OT flow. In contrast, DPF-GradNet and DPFNet-HMC maintain substantially lower and smoother flow magnitudes ($\sim 10 - 15$) across both scenarios, demonstrating that amortized and learned transport strategies produce far more controlled particle movement. IPFPF and SPFSM remain near zero, as they do not rely on explicitly learned flow steps.

Jacobian Conditioning Only DPFOT and DPFOT-HMC exhibit non-trivial Jacobian condition numbers, as they are the only models with explicit differentiable flow maps. In the low-noise setting $\left(\sigma_W^2 = 1\right)$, both models maintain condition numbers stably around 10^8, suggesting a consistently ill-conditioned transport map. Under high noise ($\sigma_W^2 = 10$), the condition numbers fluctuate wildly across six orders of magnitude $\left(10^3 - 10^9\right)$, revealing severe numerical instability in the OT-based flow. DPFNet-HMC's condition number remains near the bottom of the scale throughout, confirming that replacing the direct OT solve with a learned GradNet significantly improves the numerical conditioning of the resampling map, particularly under challenging high-noise conditions.

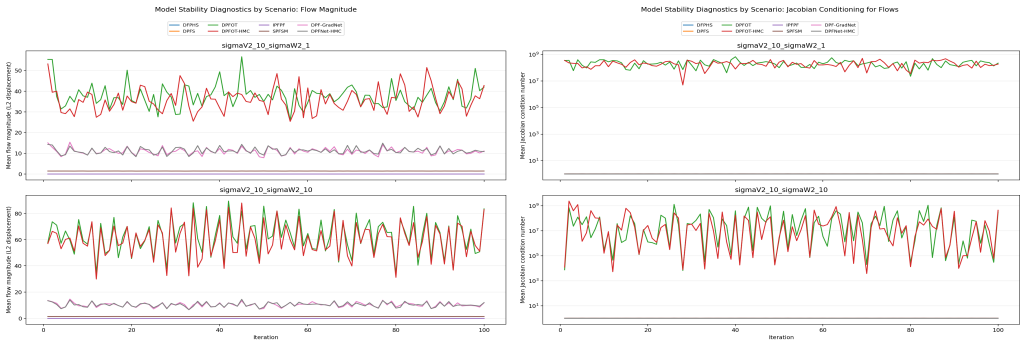

Figure 3: Flow magnitude and Jacobian conditioning for flows.
(Left: flow magnitude; Right: Jacobian conditioning)

Sensitivity in OT-based Resampling Fig. 4 and 5 illustrate the sensitivity of DPFOT and DPFOT-HMC to the Sinkhorn regularization parameter (ε) and solver iterations. For both models, the Root Mean Square Error (RMSE) is largely insensitive to the number of iterations but degrades substantially with increased regularization. This confirms that over-smoothing the transport plan harms estimation accuracy regardless of solver convergence. While HMC transitions preserve the overall sensitivity pattern, they modestly reduce the absolute RMSE under high-noise conditions ($\sigma_W^2 = 10$). Furthermore, the Effective Sample Size (ESS) remains uniformly flat across all configurations. This structural insensitivity indicates that hyperparameter tuning alone cannot resolve the severe particle collapse inherent in OT-based filters.

Computationally, runtime scales strictly with iteration count, rising sharply beyond 50 iterations, with DPFOT-HMC incurring only a marginal HMC overhead ($\sim 14\,\mathrm{s}$ versus $\sim 12\,\mathrm{s}$). Consequently, the optimal operating regime for both methods lies at low regularization ($\varepsilon \approx 0.05$–0.1) and minimal iterations (10–20). Ultimately, the failure of additional Sinkhorn iterations to yield any accuracy or diversity benefits—despite severe computational penalties—highlights the fundamental limitations of

iterative OT resampling. This strongly motivates amortized approaches such as DPF-GradNet and our proposed DPFNet-HMC, which bypass the Sinkhorn solver to enhance scalability and robustness.

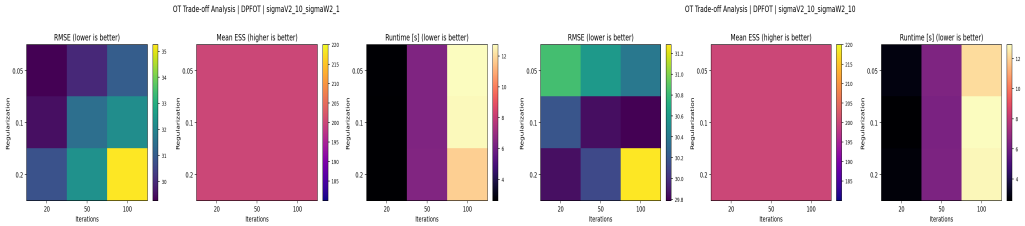

Figure 4: Trade-off for DPFOT across regularization strength and number of iterations. (Left:$\sigma_W^2 = 1$; Right: $\sigma_W^2 = 10$)

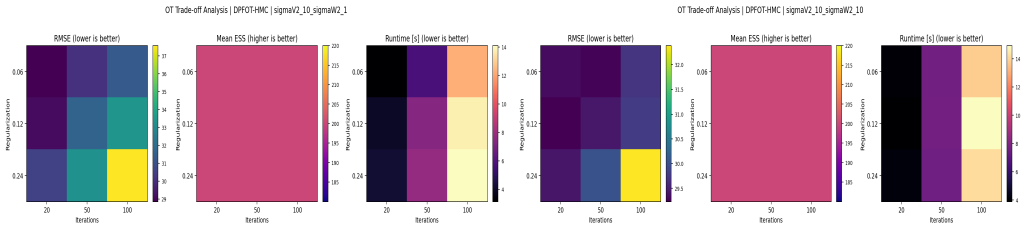

Figure 5: Trade-off for DPFOT-HMC across regularization strength and number of iterations. (Left:$\sigma_W^2 = 1$; Right: $\sigma_W^2 = 10$)

5.4 Evaluation of PMMH vs HMC

As discussed by Neal [12], Hamiltonian Monte Carlo (HMC) significantly outperforms standard random-walk Metropolis-Hastings (MH) algorithms, primarily by utilizing gradient information to avoid inefficient, diffusive exploration. Furthermore, HMC exhibits vastly superior scalability in high-dimensional spaces. Its required computation time typically scales as $\mathcal{O}(d^{5/4})$, which is substantially more efficient than the $\mathcal{O}(d^2)$ scaling of random-walk Metropolis methods.

To empirically validate these theoretical advantages within our framework, we compared the proposed DPFNet-HMC against a DPFNet-PMMH baseline under an extreme noise scenario ($\sigma_V^2 = 60$, $\sigma_W^2 = 0.2$). Consistent with theoretical expectations, DPFNet-HMC outperformed the PMMH variant, achieving a maximum relative improvement in Root Mean Square Error (RMSE) of approximately 1.57%.

5.5 Evaluation of NumPy vs TF/TFP

To justify the computational framework chosen for DPFNet-HMC, we benchmarked a representative Hamiltonian Monte Carlo (HMC) sampling workload using standard NumPy and TensorFlow/TensorFlow Probability (TF/TFP) over 5 independent trials. As shown in Table 2, TF/TFP consistently outperforms NumPy across all sequence lengths, yielding a stable speedup of approximately $1.28\times$.

Table 2: Benchmark of representative HMC sampling workload. (Results are reported as mean $\pm$ std over 5 random seeds.)

Repeats	**NumPy (s)**	**TF/TFP (s)**	**Speedup**
1,000	0.135 ± 0.003	0.106 ± 0.002	$(1.280 \pm 0.044)\times$
10,000	1.309 ± 0.012	1.020 ± 0.015	$(1.283 \pm 0.017)\times$
100,000	13.022 ± 0.106	10.204 ± 0.153	$(1.276 \pm 0.010)\times$

While a $\sim 28\%$ speedup may appear modest for a single iteration, this advantage compounds into substantial absolute time savings in particle filtering tasks, where HMC transitions are evaluated tens

or hundreds of thousands of times sequentially. Ultimately, TF/TFP proves to be the indispensable choice for two key architectural reasons. First, by using graph compilation via decorators such as `@tf.function`, TensorFlow compiles repeated-sampling loops into optimized computational graphs. This mechanism effectively amortizes the initial startup costs and drastically reduces the Python-level interpreter overhead that typically bottlenecks NumPy at scale. Second, HMC strictly requires exact log-posterior gradients to drive its leapfrog dynamics. Relying on TF/TFP integrates both the automatic differentiation engine and the probabilistic sampling kernels within a single, unified ecosystem. This unified approach inherently eliminates the severe memory-transfer bottlenecks and engineering friction that would inevitably occur if one were to alternate between NumPy for state transitions and a separate computational library for exact gradient calculations.

6 Conclusion

We introduced DPFNet-HMC, a differentiable particle filter that combines amortized transport with HMC rejuvenation. The transport stage provides a smooth alternative to discrete resampling, and the HMC step restores particle diversity by exploring the local geometry of the posterior. On a high-dimensional nonlinear benchmark, DPFNet-HMC achieves strong RMSE with stable ESS, while DPF-GradNet offers nearly the same accuracy at much lower runtime.

For production use, we recommend DPF-GradNet when runtime is the primary constraint, and DPFNet-HMC when robustness to particle collapse is critical. The main limitation of DPFNet-HMC is the need to tune HMC hyperparameters (step size and number of leapfrog steps), which can affect stability. Future work should evaluate these methods on neural state-space models and study adaptive tuning strategies for the HMC stage to reduce the need for manual calibration.

References

[1] N.J. Gordon, D.J. Salmond, and A.F.M. Smith. Novel approach to nonlinear/non-gaussian bayesian state estimation. *IEE Proceedings F Radar and Signal Processing*, 140(2):107, 1993.

[2] *Sequential Monte Carlo Methods in Practice*. Springer New York, 2001.

[3] Liyi Dai and Fred Daum. Stiffness mitigation in stochastic particle flow filters. *IEEE Transactions on Aerospace and Electronic Systems*, 58(4):3563–3577, August 2022.

[4] Rico Jonschkowski, Divyam Rastogi, and Oliver Brock. Differentiable particle filters: End-to-end learning with algorithmic priors, 2018.

[5] Steve Brooks, Andrew Gelman, Galin Jones, and Xiao-Li Meng. *Handbook of Markov Chain Monte Carlo*. Chapman and Hall/CRC, May 2011.

[6] Adrien Corenflos, James Thornton, George Deligiannidis, and Arnaud Doucet. Differentiable particle filtering via entropy-regularized optimal transport, 2021.

[7] Christophe Andrieu, Arnaud Doucet, and Roman Holenstein. Particle markov chain monte carlo methods. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 72(3):269–342, May 2010.

[8] Yunpeng Li and Mark Coates. Particle filtering with invertible particle flow. *IEEE Transactions on Signal Processing*, 65(15):4102–4116, August 2017.

[9] Adrien Corenflos, James Thornton, George Deligiannidis, and Arnaud Doucet. Differentiable particle filtering via entropy-regularized optimal transport, 2021.

[10] Rico Jonschkowski, Divyam Rastogi, and Oliver Brock. Differentiable particle filters: End-to-end learning with algorithmic priors, 2018.

[11] Shreyas Chaudhari, Srinivasa Pranav, and José M. F. Moura. Gradnetot: Learning optimal transport maps with gradnets, 2025.

[12] Radford M. Neal. Mcmc using hamiltonian dynamics. 2012.